

COMP 424 - Artificial Intelligence

Constraint Satisfaction Problems

Lecture 5: September 16th, 2026

Su Lin Blodgett (sulin.blodgett@mcgill.ca)

Jackie CK Cheung (jackie.cheung@mcgill.ca)

Announcements

- Check Ed for announcements on midterm time, A1 handout.
- Submit assignments through MyCourses!

Recap: Local search and optimization

- Two distinctions drawn compared to previously:
 - Problem: Often, interested in best state according to *evaluation function* (*optimization* problem); less interested in *path* to that state
 - Search strategy: Start with some initial solution, then employ some strategy to gradually improve it; not searching systematically
- Apply a **local search** and move in a promising direction.
 - **Hill climbing** always moves in the (locally) best direction
 - **Simulated annealing** allows some moves towards worse solutions
 - Parallelism and **beam search** can be exploited to improve results; find a better local optimum
 - **Genetic algorithms** are an alternative related to simulated annealing which has an analogy to biological evolution

Hill climbing

- Start from an initial configuration X_0 with value $E(X_0)$
 $X \leftarrow X_0$, and $E \leftarrow E(X_0)$
- Repeat until satisfied:
 - Generate the set of neighbours of X_i and their value $E(X_i)$.
 - Let $E_{max} = \max_i E(X_i)$ be the value of the **best** neighbour,
 $i^* = \operatorname{argmax}_i E(X_i)$ be the index of the **best** neighbour.
 - if $E_{max} \leq E$:
return X (we are at an optimum)
 - else:
let $X \leftarrow X_{i^*}$, and $E \leftarrow E_{max}$. (take a greedy step)
- Can get stuck in a **local maximum** or **plateau**!

Simulated annealing

- Idea: allow some "bad" moves to try to escape local maxima
 - Decrease size and frequency of "bad moves" over time
- Start from an initial configuration X_0 with value $E(X_0)$.
 $X \leftarrow X_0$, and $E \leftarrow E(X_0)$
- Repeat until satisfied:
 - Let X_i be a *random neighbour* of X with value $E(X_i)$.
 - If $E_i > E$, let $X_i \leftarrow X$ and let $E \leftarrow E_i$ (we found a new better solution).
 - Else, *with some probability p* , still accept the move: $X \leftarrow X_i$ and $E \leftarrow E_i$.
- What value to use for p ?
 - $p = e^{-(E-E_i)/T}$
 - We gradually decrease the value of the temperature T across iterations.

Local beam search

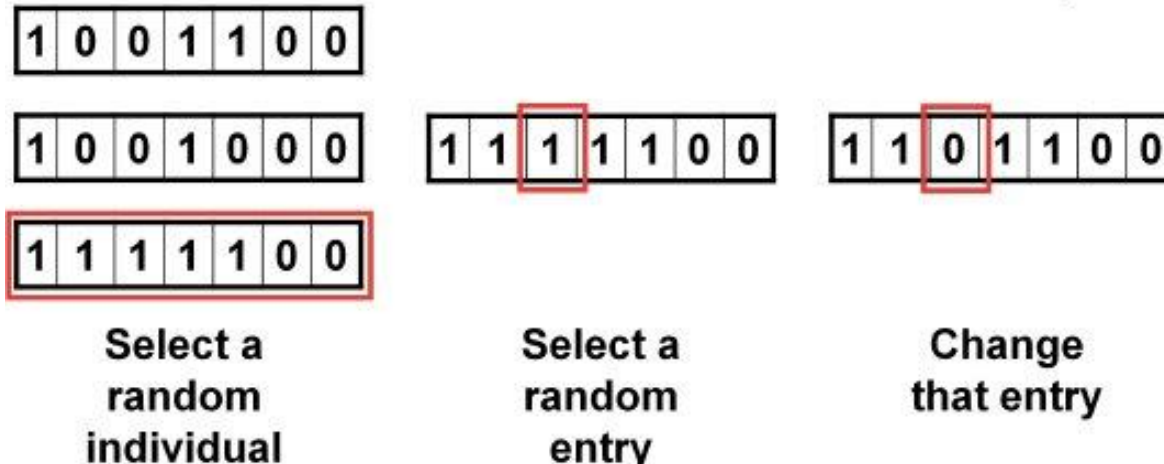
- Keep track of k states rather than just one state
- Algorithm:
 - Begin with k randomly generated states
 - At each step, generate all the neighbours of the k states
 - If one is a goal, terminate and return that state
 - Otherwise, keep the top k solutions across all the neighbours, discard the remaining
 - k is called the **beam width**

Genetic algorithms

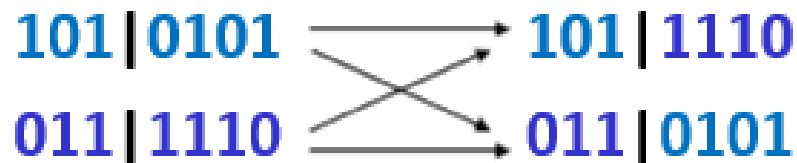
- A candidate solution is called an **individual**.
 - In a traveling salesman problem, an individual is a tour
- Each individual has a **fitness** scored by a **fitness function**.
- A set of individuals is called a **population**.
- Populations change over **generations**, by applying **operations** to individuals.
 - **operations** = {mutation, crossover, selection}
 - probability of selection is proportional to fitness score
- Individuals with higher fitness are more likely to survive & reproduce.
- Individual can be represented by a binary string:
 - Allows operations to be carried out easily.

Mutation and crossover

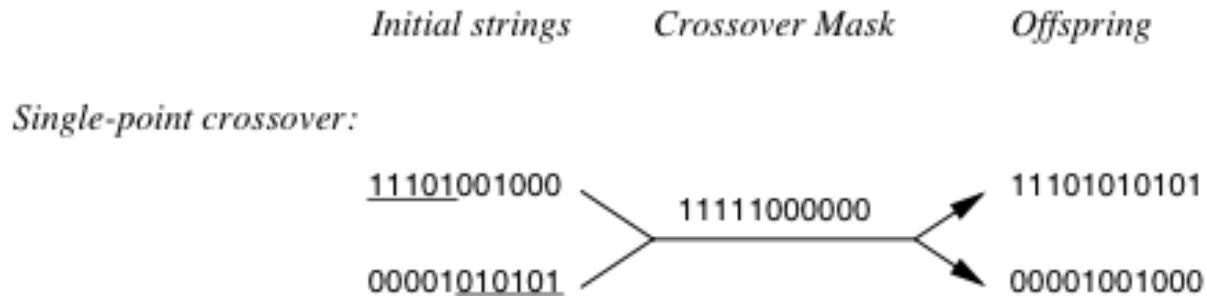
- Mutation: a way to generate desirable features by injecting random change; controlled by mutation rate



- Crossover: combine parts of individuals to create new ones
 - Single-point crossover: choose a crossover point, cut individuals there, swap the pieces



Encoding operators as binary masks

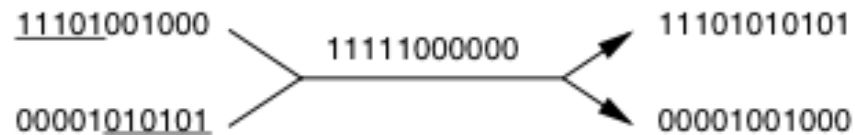


- Implementation:
 - Use a crossover mask, which is a binary string
At position i , take from first parent if $mask_i$ is 1, otherwise take from other parent

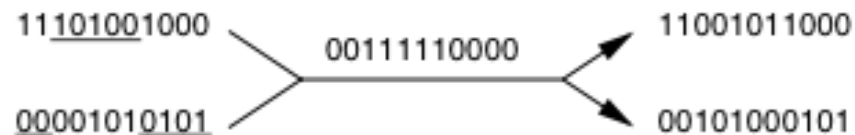
Encoding operators as binary masks

Initial strings Crossover Mask Offspring

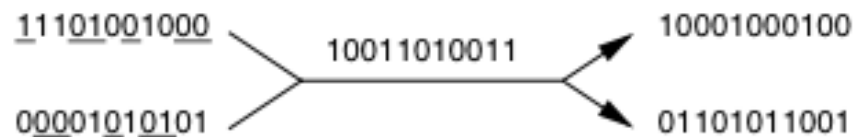
Single-point crossover:



Two-point crossover:



Uniform crossover:



Typical genetic algorithm

function GENETIC-ALGORITHM(*population*, FITNESS-FN) **returns** an individual

inputs: *population*, a set of individuals

FITNESS-FN, a function that measures the fitness of an individual

repeat

new_population $\leftarrow$ empty set

for $i = 1$ **to** SIZE(*population*) **do**

$x \leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

$y \leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

$child \leftarrow$ REPRODUCE(x, y) i.e., crossover

if (small random probability) **then** $child \leftarrow$ MUTATE($child$)

 add $child$ to *new_population*

population $\leftarrow$ *new_population*

until some individual is fit enough, or enough time has elapsed

return the best individual in *population*, according to FITNESS-FN

Selection: Survival of the fittest

- As in natural evolution, fittest individuals are more likely to survive.

- Several ways to implement this idea:

1. Fitness proportionate selection:

$$P(i) = \frac{Fitness(i)}{\sum_{j=1}^p Fitness(j)}$$

Can lead to crowding (multiple copies being propagated).

2. Tournament selection:

Pick i, j at random with uniform probability. With prob p select the fitter one. Only requires comparing two individuals.

3. Rank selection:

Sort all individuals by fitness. Probability of selection is proportional to rank.

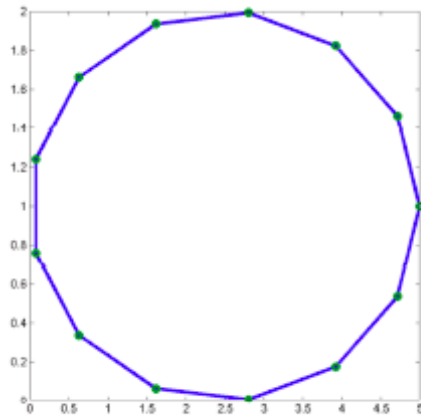
4. Softmax (Boltzmann) selection: $P(i) = \frac{e^{Fitness(i)/T}}{\sum_{j=1}^p e^{Fitness(j)/T}}$

- **Elitist variant:** Copy the best individual(s) directly to the next generation

Example: Solving TSP with a GA

- Each individual is a tour.
- Mutation swaps a pair of edges (many other operations are possible and have been tried in literature.)
- Crossover cuts the parents in two and swaps them. Reject any invalid offsprings.
- Fitness is the length of the tour.
- Note that GA operations (crossover and mutation) described here are fancier than the simple binary examples given before.

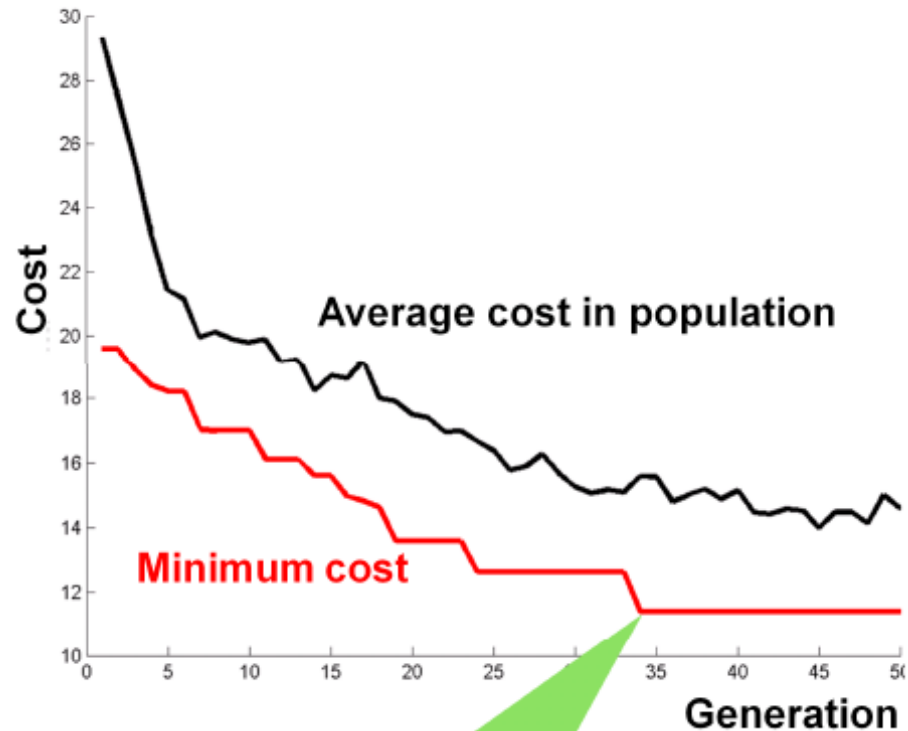
Example: Solving TSP with a GA



$N = 13$

$P = 100$ elements in population

$\mu = 4\%$ mutation rate
 $r = 50\%$ reproduction rate



Optimal solution reached at generation 35

The good and bad of GAs

- Good:
 - Intuitively appealing, due to evolution analogy.
 - If tuned right, can be very effective (good solution with few steps).
- Bad:
 - Performance depends crucially on the problem encoding. Good encodings are difficult to find!
 - Many parameters to tweak! Bad parameter settings can result in very slow progress, or the algorithm is stuck in local minima.
 - With mutation rate is too low, can get overcrowding (many copies of the identical individuals in the population).

Today: Specifying search problems

- How do we actually formulate practical problems as search?
 - *I have 5 midterms in the space of two weeks. The one for COMP 424 is before the one for COMP 551, but COMP 551 is pretty hard so I need to start studying in advance, because I cannot study more than 5 hours per course per week, or 8 hours in total across all the courses...*
- **Today's key idea:** specify problems **declaratively** as constraints
 - We list what we know, not how to solve a problem!

Searching with constraints

- Many other interesting problems have strict constraints:
E.g. Must visit city A (to re-supply) before visiting city B (to sell).
- How can we incorporate this information in the search process?
 - At a minimum, ensure the search will be limited to solutions that respect the constraints. Sometimes very few “legal solutions”.
 - Even better, *use the constraints to narrow the search space*.

Example

- Colour a map so that no adjacent territories have the same colour.



Constraint satisfaction problems (CSPs)

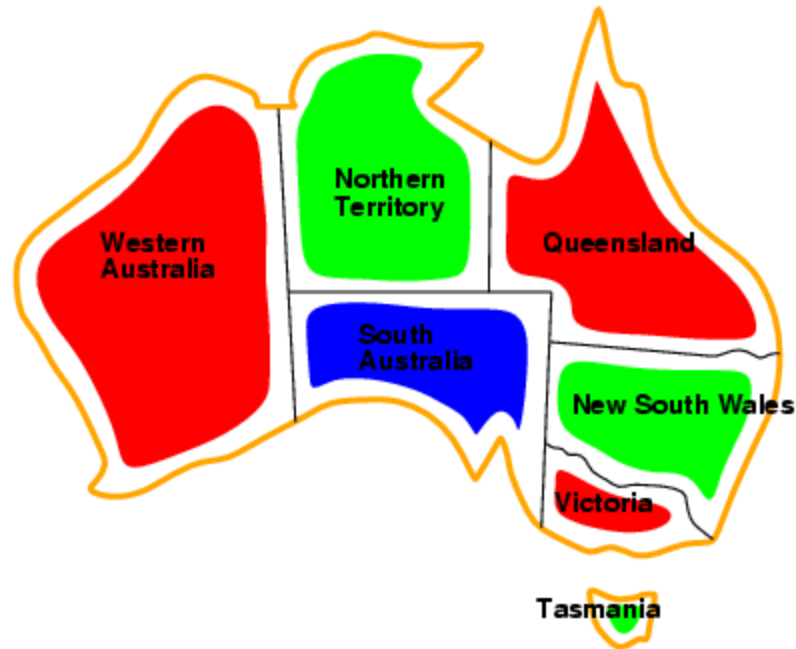
- A **CSP** is defined by:
 - Set of **variables** V_i , that can take values from domain D_i
 - Set of **constraints** specifying what combinations of values are allowed (for subsets of variables, e.g., pairs of variables)
 - Constraints can be represented:
 - Implicitly, as a function, testing for the satisfaction of the constraint.
E.g. $C_1 \neq C_2$
 - Explicitly, as a list of allowable values. E.g. $(C_1=R, C_2=G), (C_1=G, C_2=R), (C_1=B, C_2=R), \dots$
- A **CSP solution** is an assignment of values to all variables that does not violate any constraints.
- We typically want to find **any solution** or find that there is **no solution**.

Example



- **Variables** WA, NT, Q, NSW, V, SA, T
- **Domains** $D_i = \{\text{red, green, blue}\}$
- **Constraints:** adjacent regions must have different colors
- E.g., $WA \neq NT$

Example



- Solutions are complete and consistent assignments.
- E.g., WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green

Sudoku

- Every row, column, 3x3 block must contain exact one instance of a number from 1 to 9.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Initial state

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Solution

Types of variables

Many possibilities

- Boolean variables (e.g., satisfiability)
- Finite domain, discrete variables (e.g., colouring)
- Infinite domain, discrete variables (e.g., integers)
- Continuous variables (e.g., reals)

Problem complexity? Ranges from solvable in poly-time (e.g., linear programming) to NP-complete to undecidable.

Types of constraints

- **Unary:** involve 1 variable
 - e.g., $WA \neq \text{green}$
- **Binary:** involve 2 variables
 - e.g., $WA \neq NT$
- **Higher-order:** involve ≥ 3 variables
 - e.g., $\text{Between}(X, Y, Z)$
- **Relations:**
 - $T_1 + d_1 \leq T_2$ (Task₂ has to come after Task₁) Scheduling
 - $\text{Alldiff}(\text{variables in a row}), \text{Alldiff}(\text{variables in a column}),$
 $\text{Alldiff}(\text{variables in a 3x3 square})$ Sudoku
- **Preferences (soft constraints):** can be represented using costs and lead to constrained optimization problems.
 - e.g., morning vs. afternoon preferences in exam scheduling

Real-world CSPs

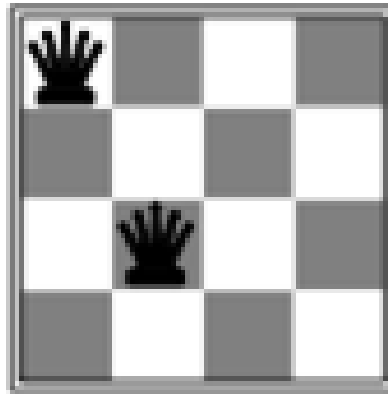
Often involves allocating *limited* resources:

- Timetable problems (e.g. which class is offered when, where)
- Hardware configuration
- Transportation scheduling, factory scheduling
- Puzzle solving (crosswords, Sudoku)

Example: 4 queens problem

- Put 4 queens on 4x4 board so that none attack each other:

Partial assignment:



- Formulate this as a CSP by defining:
 - Variables and domains
 - Constraints

4 Queens Problem

Put one queen per column. Let value indicate row of each queen.

- Variables: $\{Q_1, Q_2, Q_3, Q_4\}$
- Domain (same for all variables): $\{1, 2, 3, 4\}$
- Constraints:
 - $Q_i \neq Q_j$ (cannot be in same row)
 - $|Q_i - Q_j| \neq |i - j|$ (cannot be in same diagonal)
- Can also translate each constraint into set of allowable values for its variables:
 - Values for (Q_1, Q_2) : $(Q_1=1, Q_2=3), (Q_1=1, Q_2=4), (Q_1=2, Q_2=4)$ etc.

Example: Sudoku

- Formulate sudoku as a CSP

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Sudoku as a CSP

- Variables?
- Constraints?

Sudoku as a CSP

5	3	
6		
	9	8

- Variables
 - $\text{Cell}_{i,j}$ for row i from $[1, 9]$, and col j from $[1, 9]$
 - Domain: $\{1, \dots, 9\}$ (same for all variables)
- Constraints
 - $\text{Cell}_{1,1} = 5$; $\text{Cell}_{1,2} = 3$; etc. (filling in all known values)
 - $\text{Alldiff}(\text{Cell}_{1,j} \text{ for all indices } j)$; $\text{Alldiff}(\text{Cell}_{2,j} \text{ for all indices } j)$; etc.
 - $\text{Alldiff}(\text{Cell}_{i,1} \text{ for all indices } i)$; $\text{Alldiff}(\text{Cell}_{i,2} \text{ for all indices } i)$; etc.
 - (To specify Alldiff):
 $|\text{uniqueset}(\text{Cell}_{1,j} \text{ for all indices } j)| = |\text{Cell}_{1,j} \text{ for all indices } j|$
 - For each 3x3 subtable with cells C:
 - $\text{Alldiff}(C)$

Overview of approaches for solving CSPs

- **Constructive approach**

- State is defined by the set of values assigned so far
- Apply forward search to fill the solution
- General-purpose algorithm which works for all CSPs

- **Local search approach**

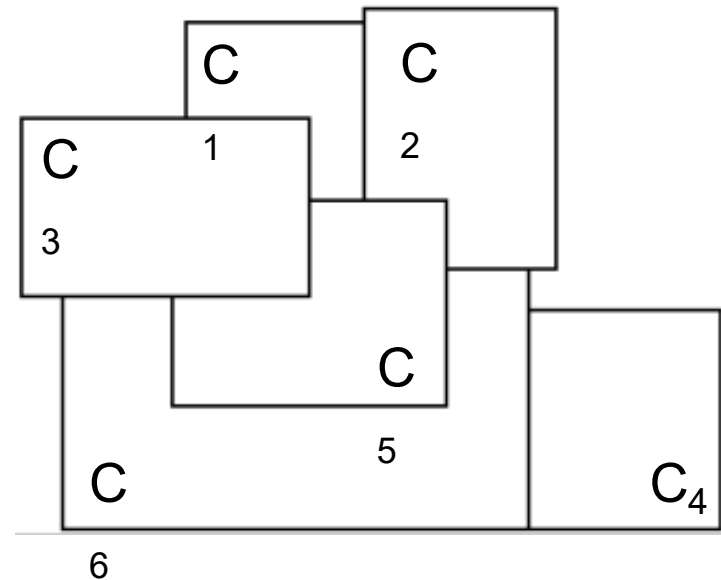
- Start with a broken but complete assignment of values to variables
- Gradually fix broken constraints by re-assigning variables
- Use optimization approaches to decrease # broken constraints (hill-climbing, simulated annealing)

Constructive search for CSPs

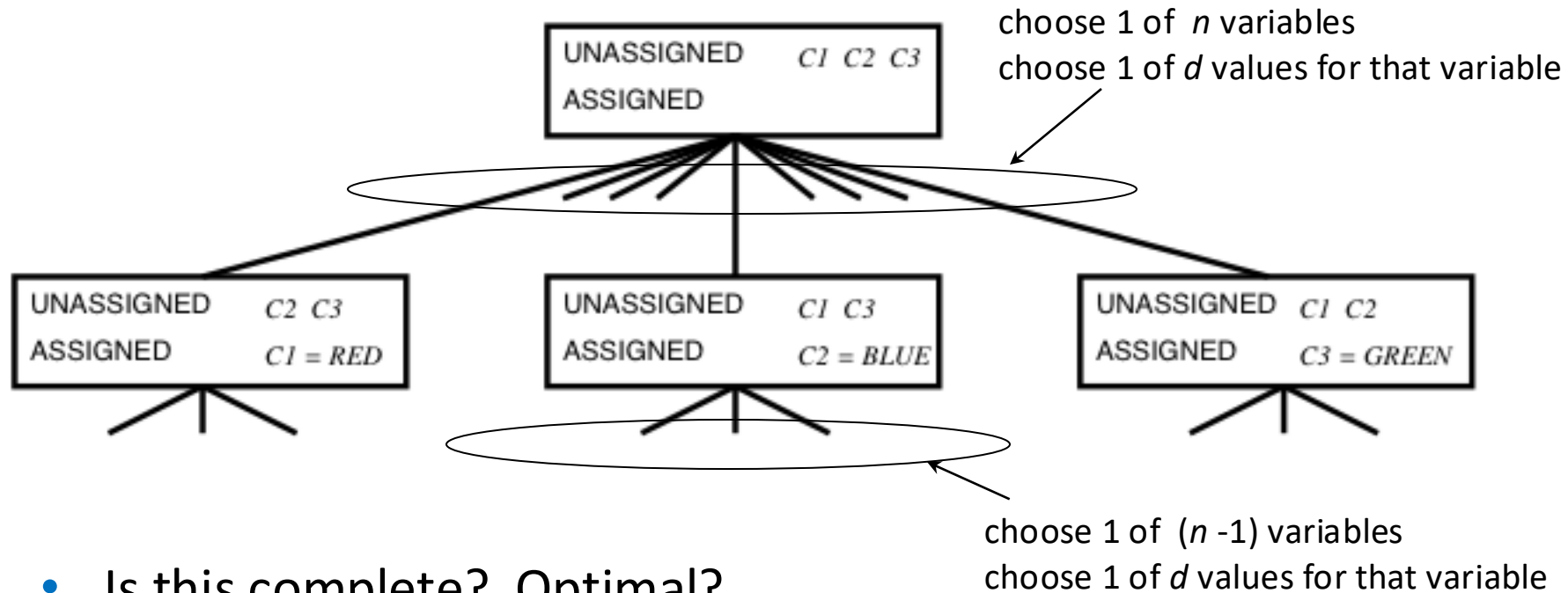
- Problem definition:
 - **State**: defined by set of values assigned so far, could be *partial* and/or *inconsistent* assignment
 - **Initial state**: all variables are unassigned.
 - **Operators**: assign a value to an unassigned variable.
 - **Goal test**: all variables assigned, no constraint violated.
i.e., *complete* and *consistent* assignment
- Problem has deterministic action, fully observable state.
Important observation: Depth is limited to the number of variables, n .
So we can apply DFS (or depth-limited search)

Example

- Color abstract map so that adjacent countries don't same color.
 - Variables: Countries C_i
 - Domains: {Red, Blue, Green}
 - Constraints: $\{C_1 \neq C_2, C_1 \neq C_5, \dots\}$



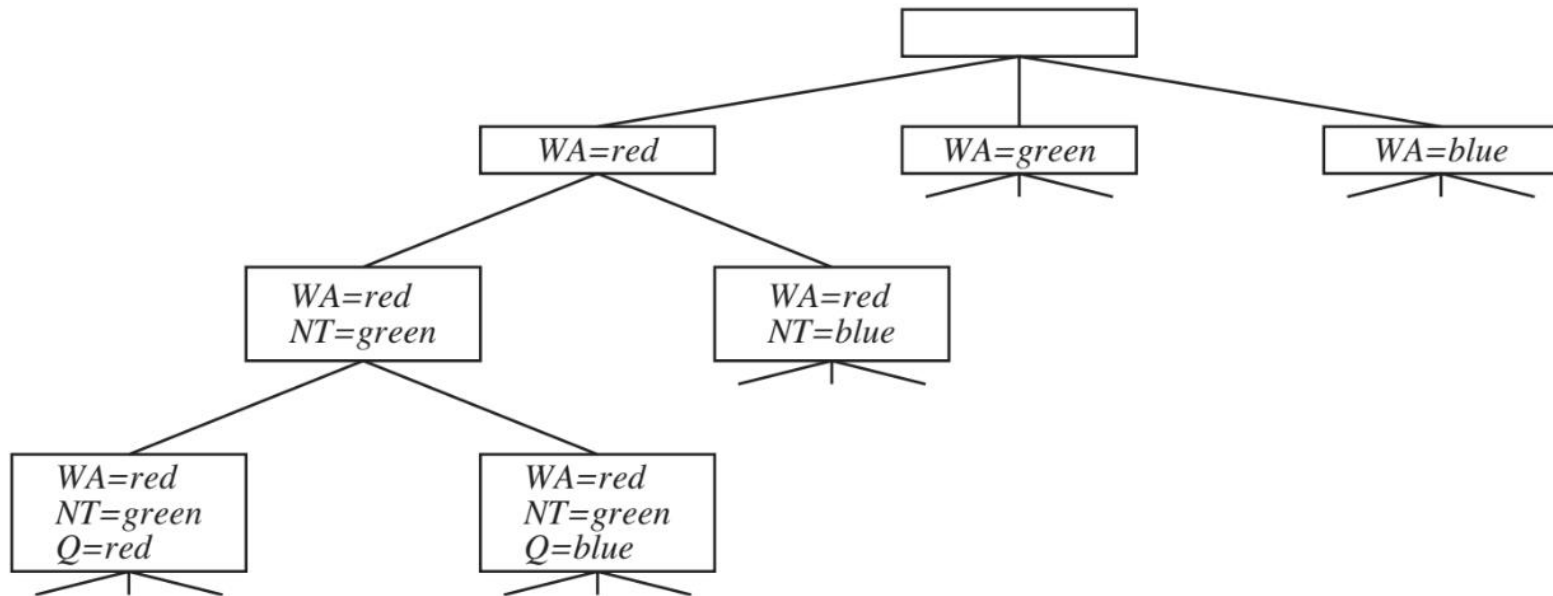
Standard uninformed search for map coloring



- Is this complete? Optimal?
- Is this a practical approach? What is the complexity?

Standard uninformed search for map coloring

choose 1 of n variables
choose 1 of d values for that variable



Yes: known (not infinite) solution depth. Yes: If we check the constraints.

- Is this a practical approach? What is the complexity?
 $[n \times d] \times [(n-1) \times d] \times [(n-2) \times d] \times \dots \times [2 \times d] \times d = n! d^n$

Analysis of the simple approach

Branching factor is very high:

$\sum_i d_i$ (i sums over unassigned variables)

BUT: There can be only d^n unique complete assignments.

More important observations:

- Order in which variables are assigned is irrelevant -> Many paths are equivalent!
- Adding assignments cannot correct a violated constraint!

Backtracking search

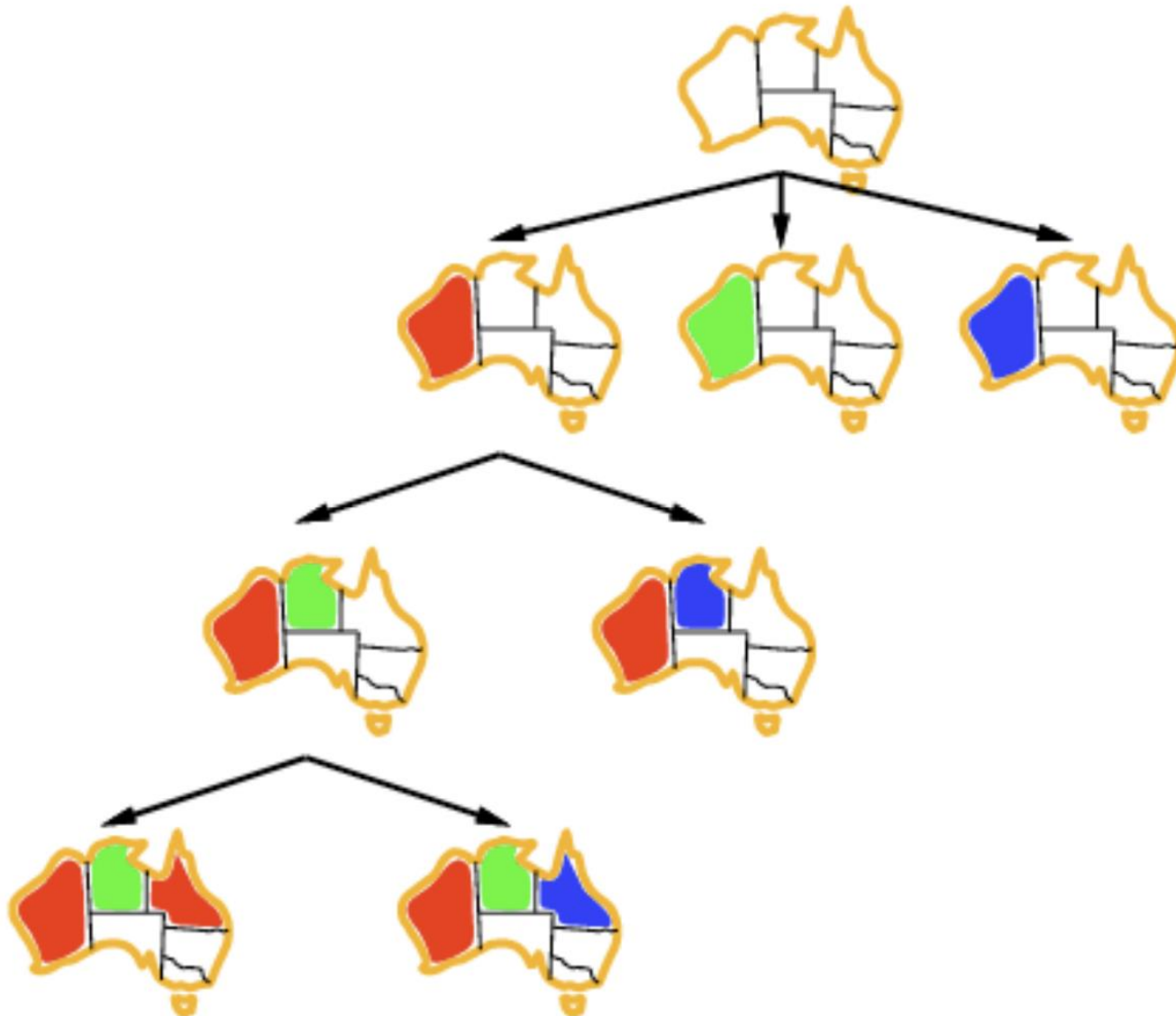
At each step, pick a remaining unassigned variable at each level of search tree, then pick a value for that variable

- i.e., consider only assignments to a single variable at each node
- This way, $b = |D_i|$

Algorithm:

- Select an unassigned variable, X .
- For each $value = \{x_1, \dots, x_n\}$ in the domain of that variable
 - If the value satisfies the constraints, let $X = x_i$ and exit the loop.
- If an assignment was found, move to the next variable.
- If no assignment, go back to preceding variable and try different value.
- This is the basic uninformed algorithm for CSPs.
 - Can solve n-queens for $n \approx 25$.

Backtracking search



Backtracking search

At each step, pick a remaining unassigned variable at each level of search tree, then pick a value for that variable

- i.e., consider only assignments to a single variable at each node
- This way, $b = |D_i|$

Algorithm:

- Select an unassigned variable, X .
- For each $value = \{x_1, \dots, x_n\}$ in the domain of that variable
 - If the value satisfies the constraints, let $X = x_i$ and exit the loop.
- If an assignment was found, move to the next variable.
- If no assignment, go back to preceding variable and try different value.

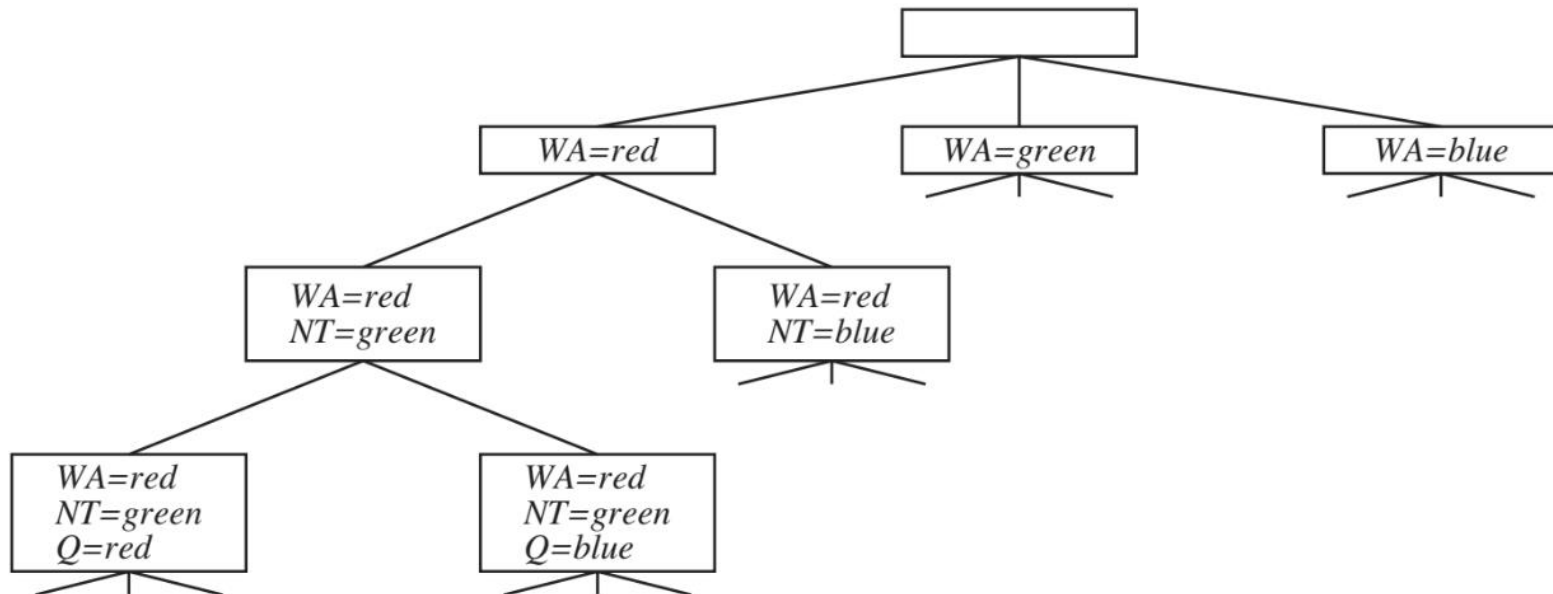
Let's focus on how we order the variables and the values for each variables

Selecting a variable



Heuristics:

- 1. Minimum-remaining values:** Choose the variable that is the most constrained (i.e., fewest legal values).
 - e.g., after WA = red and NT = green, assign SA instead of Q



Selecting a variable



Heuristics:

1. **Minimum-remaining values:** Choose the variable that is the most constrained (i.e., fewest legal values).
2. **Degree heuristic:** Choose the variable that is involved in the largest number of constraints on other unassigned values
 - Use this to break ties from Minimum-remaining value heuristic
 - e.g., choosing the first region to colour in Australia

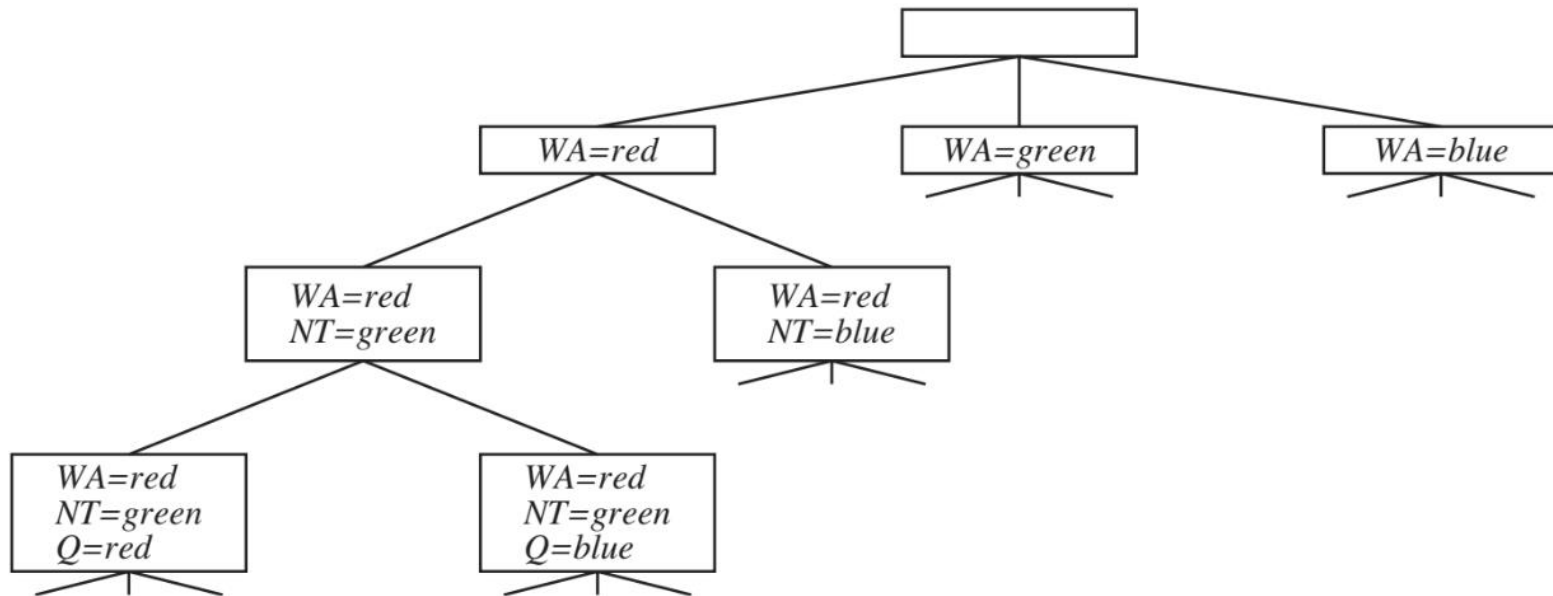
Selecting a value

- Heuristic:
 - **Least-constraining value:** Assign the value that rules out the fewest values for other variables
- Note that we want the *variable* that is the *most* constrained with the fewest remaining values (to quickly rule out many possible solutions), but the *value* that is *least* constraining, to offer flexibility to find some solution.

Selecting a value



- Heuristic:
 - **Least-constraining value:** Assign the value that rules out the fewest values for other variables
 - e.g., after WA = red and NT = green, prefer Q = red over Q = blue



How to solve CSPs?

General strategies:

- **Search**
 - Constructive methods (e.g., backtracking search)
 - Local search
- **Inference**
 - Prune the domains of the variables by using the constraints that those variables are involved in

Recall example

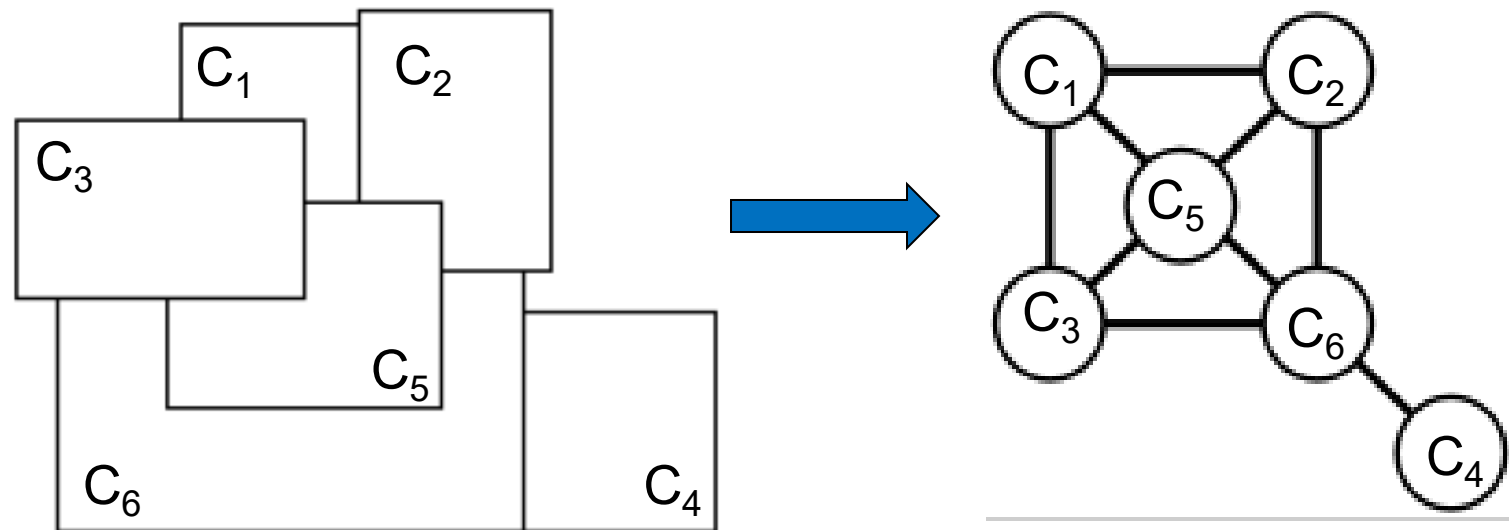
- Colour a map so that no adjacent territories have the same colour.



Data structure: Constraint graph

- Nodes are variables, arcs show constraints
- Graph structure can be exploited to accelerate solution search

E.g. Map colouring:



Using constraint graph

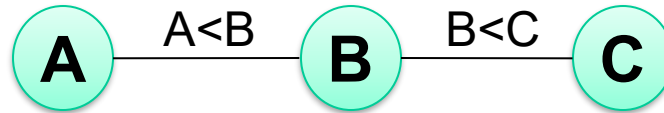
- Perform **inference** using constraint graph in order to reduce search space
- **Idea:** Pre-process the graph before running a search algorithm to remove obvious inconsistencies
- e.g., Two variables: A, B
 $domain(A) = domain(B) = \{1, 2, 3\}$
Constraint: $A < B$
Inference step: *A cannot be 3 and B cannot be 1*

Arc consistency

- A variable is **arc consistent** if every value in its domain satisfies that variable's binary constraints (i.e., those with one other variable).
 - i.e., X_i is arc-consistent w.r.t. X_j if for every value in D_i , there is some value in D_j that satisfies the constraint on (X_i, X_j)
- A variable is **generalized arc consistent** if every value in its domain satisfies that variable's n -ary constraints (with $n-1$ other variables).
- A network is **arc consistent** if all of its variables are simultaneously arc consistent.

Example: Consistency constraints

A CSP with variables A , B , C , each with domain $\{1, 2, 3, 4\}$:



After some pruning:

$A=\{1, 2, 3\}$; $B=\{2, 3\}$; $C=\{2, 3, 4\}$

but network is still not arc consistent! (Check $A < B$ again)

Arc consistent variable assignment:

$A=\{1, 2\}$; $B=\{2, 3\}$; $C=\{3, 4\}$

Possible outcomes after inference

What do the following outcomes after pruning mean?

1. At least one variable's domain is now the empty set.
2. Every variable's domain has exactly one remaining value left.
3. Every variable's domain has one or more remaining values left (and we're not in case 2).

AC-3 Algorithm

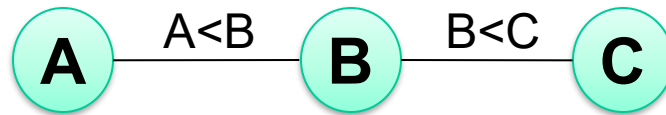
- Given a CSP (with binary constraints), perform inference to make the associated constraint graph arc consistent.

AC-3 (CSP) :

```
worklist ← all arcs in the CSP's constraint graph
while worklist is not empty:
    (X, Y) ← worklist.pop()
    prune dom(X) wrt constraints involving Y
    if size(dom(X)) == 0: no solution
    if dom(X) was changed:
        for N in neighbours of X except Y:
            worklist.push((N, X))
done [start running search algorithm if necessary]
```

Example

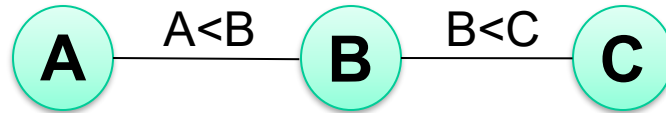
A CSP with variables A, B, C, each with domain {1, 2, 3, 4}:



Let's run AC-3 on this problem.

Example details

A CSP with variables A, B, C, each with domain {1, 2, 3, 4}:



Domains

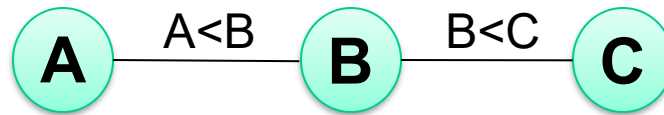
A:	1	2	3	4
B:	1	2	3	4
C:	1	2	3	4

Worklist: { (A,B) (B,A) (B,C) (C,B) }



Example result

A CSP with variables A, B, C, each with domain {1, 2, 3, 4}:



Domains

A:	1	2	3	4
B:	1	2	3	4
C:	1	2	3	4

Worklist: { (A,B) (B,A) (B,C) (C,B) (A,B) }



Notes on AC-3

- If there is an arc between (X, Y) in the CSP's constraint graph, need to add both (X, Y) and (Y, X) to the worklist.
 - Potentially need to reduce both $\text{dom}(X)$ and $\text{dom}(Y)$
- Time complexity: $O(ed^3)$
 - e is number of edges
 - d is maximum size of domain across variables
 - Each edge can be added $O(d)$ times, because an edge is added when a value is deleted, $O(ed)$ in total
 - Each check for a binary constraint takes $O(d^2)$

Interleaving search and inference

- For very large problems, may not want to do inference on the whole CSP all at once at the start.
- What if we interleaved search and inference?
 - Take a step in search
 - Then perform some local inference to prune some variables' domains
 - Then continue search
 - ...

Forward checking

Idea: Keep track of legal values for unassigned variables.

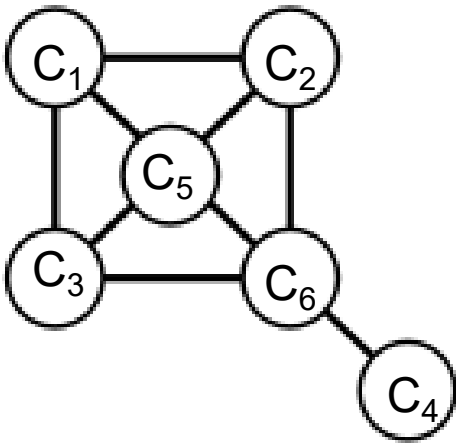
- When you assign a variable X
 - look at each unassigned variable Y connected to X (by a constraint)
 - delete from Y 's domain any value that is inconsistent with the value of X
- Can solve n -queens for $n \approx 30$.

Forward checking

Idea: Keep track of legal values for unassigned variables.

- When you assign a variable X
 - look at each unassigned variable Y connected to X (by a constraint)
 - delete from Y's domain any value that is inconsistent with the value of X

E.g. Map colouring



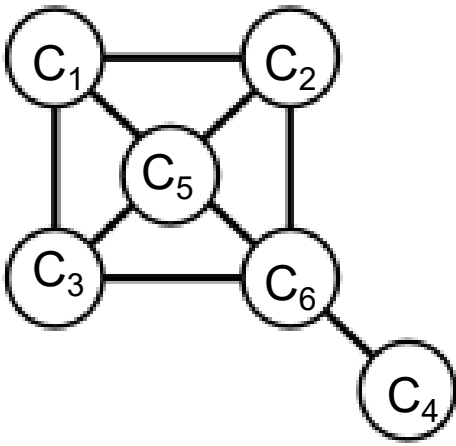
	Nothing assigned		
C1	RGB		
C2	RGB		
C3	RGB		
C4	RGB		
C5	RGB		
C6	RGB		

Forward checking

Idea: Keep track of legal values for unassigned variables.

- When you assign a variable X , establish arc consistency for it
 - look at each unassigned variable Y connected to X (by a constraint)
 - delete from Y 's domain any value that is inconsistent with the value of X

E.g. Map colouring



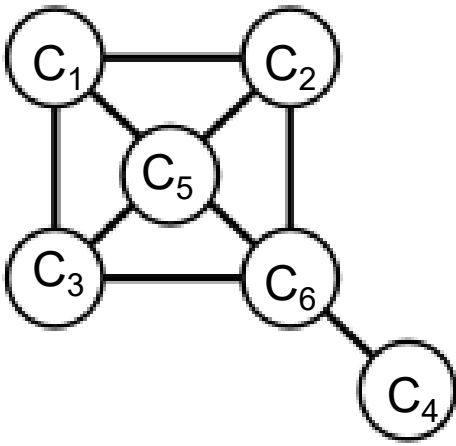
	Nothing assigned	Assign C1 = Red	
C1	RGB	R	
C2	RGB	GB	
C3	RGB	GB	
C4	RGB	RGB	
C5	RGB	GB	
C6	RGB	RGB	

Forward checking

Idea: Keep track of legal values for unassigned variables.

- When you assign a variable X , establish arc consistency for it
 - look at each unassigned variable Y connected to X (by a constraint)
 - delete from Y 's domain any value that is inconsistent with the value of X

E.g. Map colouring



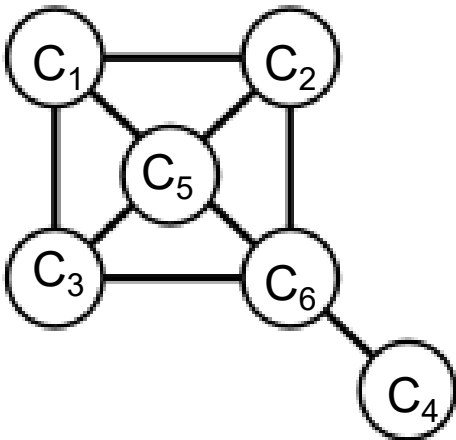
	Nothing assigned	Assign C1 = Red	Assign C2 = G
C1	RGB	R	R
C2	RGB	GB	G
C3	RGB	GB	GB
C4	RGB	RGB	RGB
C5	RGB	GB	B by forward checking!
C6	RGB	RGB	RB

Forward checking

Idea: Keep track of legal values for unassigned variables.

- When you assign a variable X , establish arc consistency for it
 - look at each unassigned variable Y connected to X (by a constraint)
 - delete from Y 's domain any value that is inconsistent with the value of X

E.g. Map colouring



	Nothing assigned	Assign C1 = Red	Assign C2 = G
C1	RGB	R	R
C2	RGB	GB	G
C3	RGB	GB	GB
C4	RGB	RGB	RGB
C5	RGB	GB	B by forward checking!
C6	RGB	RGB	RB

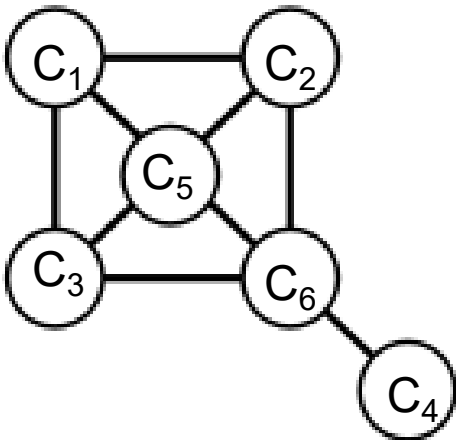
Can also apply **AC-3** to get further pruning!

Forward checking

Idea: Keep track of legal values for unassigned variables.

- When you assign a variable X , establish arc consistency for it
 - look at each unassigned variable Y connected to X (by a constraint)
 - delete from Y 's domain any value that is inconsistent with the value of X

E.g. Map colouring



	Nothing assigned	Assign C1 = Red	Assign C2 = G
C1	RGB	R	R
C2	RGB	GB	G
C3	RGB	GB	GB G
C4	RGB	RGB	RGB
C5	RGB	GB	B
C6	RGB	RGB	RB R

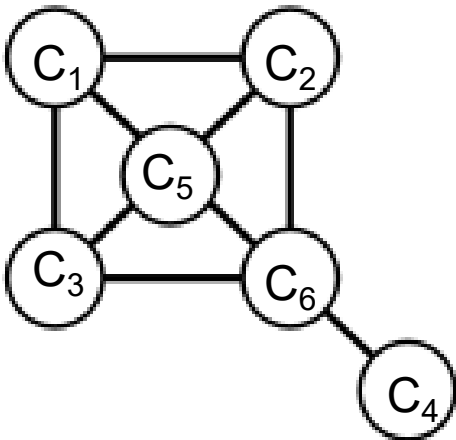
Can also apply **AC-3** to get further pruning!

Forward checking

Idea: Keep track of legal values for unassigned variables.

- When you assign a variable X , establish arc consistency for it
 - look at each unassigned variable Y connected to X (by a constraint)
 - delete from Y 's domain any value that is inconsistent with the value of X

E.g. Map colouring



	Nothing assigned	Assign C1 = Red	Assign C2 = G
C1	RGB	R	R
C2	RGB	GB	G
C3	RGB	GB	GB G
C4	RGB	RGB	RGB GB
C5	RGB	GB	B
C6	RGB	RGB	RB R

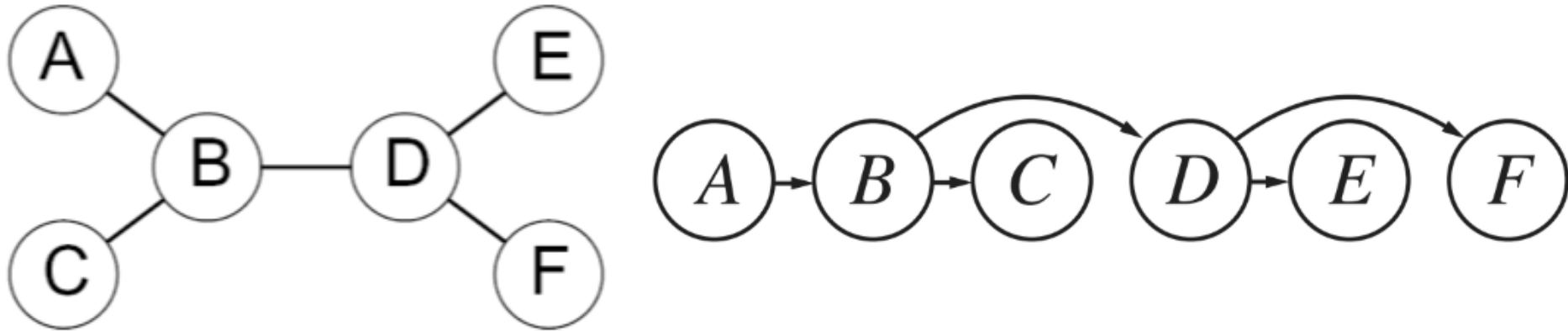
Can also apply **AC-3** to get further pruning!

Taking advantage of problem structure

- Worst-case complexity is d^n (where d is the number of possible values and n is the number of variables).
- But a lot of problems are much easier!
- Disjoint components can be solved independently
- Trees can be solved inexpensively

Taking advantage of problem structure

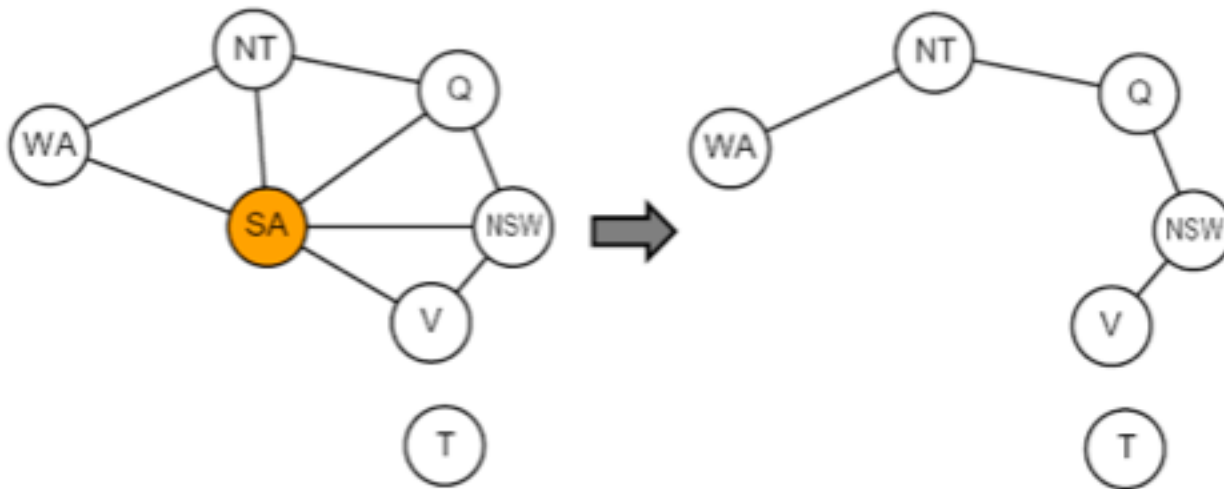
- Trees can be solved inexpensively
- Creating trees: topological sort



- Slightly different definition of consistency: **directed arc consistency**
 - Under an ordering of variables $X_1, X_2, \dots, X_n$, DAC iff every X_i is arc-consistent with each X_j for $j > i$
- Tree-structured constraint graph: complexity is $O(nd^2)$

Taking advantage of problem structure

- Nearly-tree structured graph: complexity is $O(d^c(n-c)d^2)$ using **cutset conditioning**:
 - Find a set of variables which, when removed, turn graph into tree.
 - Instantiate them all possible ways. Good if c (size of cutset) is small.



Key insight: Leverage structure to accelerate solving!

How to solve CSPs?

General strategies:

- **Search**
 - Constructive methods (e.g., backtracking search)
 - **Local search**
- **Inference**
 - Prune the domains of the variables by using the constraints that those variables are involved in

Local search for CSPs

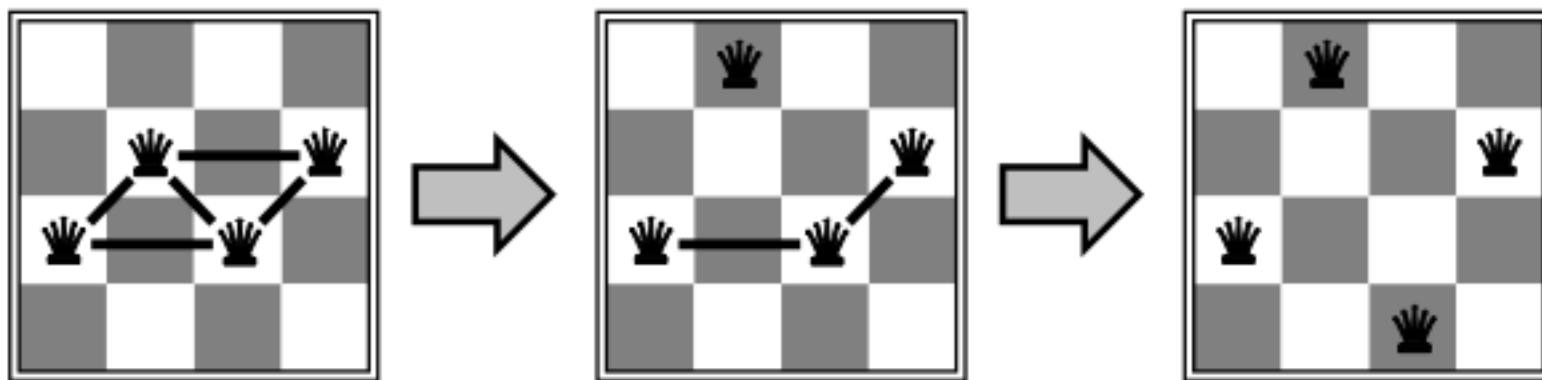
General idea: Iterative improvement algorithm

- Start with a broken but complete assignment of values to variables.
- Allow variable assignments that don't satisfy some constraints.
- Randomly select any conflicted variables.
- Operators reassign variable values.
 - **Min-conflicts heuristic** chooses value that results in the fewest number of constraint violations.

a.k.a. Hill-climbing optimization! (Could also use simulated annealing.)

Example: 4-Queens

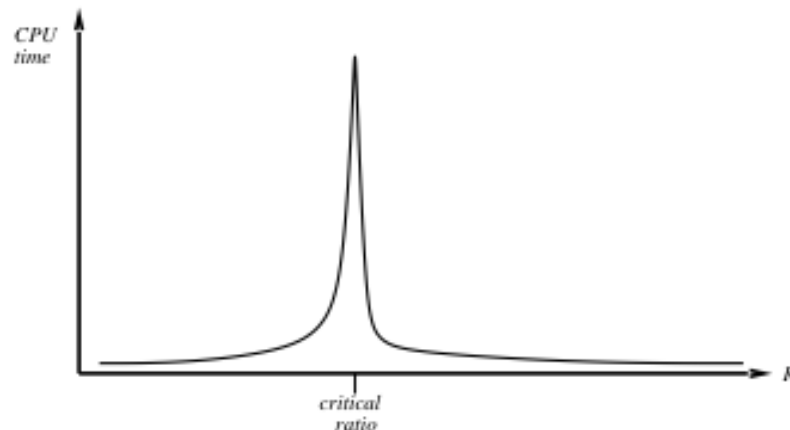
- States: 4 queens in 4 columns ($4^4 = 256$ states)
- Operators: move queen in column
- Goal test: no attacks
- Evaluation function: number of attacks



Performance of min-conflicts heuristic

- Given random initial state, can solve n -queens in almost constant time for arbitrary n with high probability (e.g. $n=10^7$).
- The same appears to be true for many randomly-generated CSPs, except in a narrow range of the ratio:

$$R = \frac{\text{number of constraints}}{\text{number of variables}}$$



Summary

- CSPs are everywhere. Be able to recognize them!
- Know how to cast CSP solving as a search problems
- Understand basic concepts: constraint graph, arc consistency
- Understand both constructive and iterative improvement methods to solve CSPs
- Know how to apply the various heuristics:
 - minimum-remaining-values, least-constraining value, degree
- Iterative improvement methods using min-conflict heuristic are very general and often work better